Haskish Programming Guide

A Comprehensive Introduction to Functional Programming

This booklet should be used in conjunction with the Haskish interactive learning environment.

Visit: https://www.korovatron.co.uk/haskish/

All lessons are also available from the main menu by accessing the Lessons section.

Table of Contents

1. Using the REPL
2. Variables
3. Simple Functions
4. Partial Function Application
5. First-Class Functions
6. Function Composition
7. Using the Function Panel
8. Building More Complex Programs
9. Introduction to Lists
10. Pattern Matching & Recursion
11. Guards & Conditionals
12. Higher-Order Functions: Map, Filter, and Fold
13. List Comprehensions
14. Tuples and Destructuring
15. Lambda Functions

Using the REPL

Get familiar with the Interactive ***REPL*** (**R**ead-**E**val-**P**rint **L**oop). The REPL shows results immediately.

> ### Important: Characters vs Strings
>
> In Haskish (like Haskell), there's an important distinction:
>
> - **Single quotes** `'a'` create a **Char** (a single character)
> - **Double quotes** `"abc"` create a **String** (a list of characters)
>
> So `"hello"` is actually the same as `['h','e','l','l','o']` - a list of individual characters!
>
> This is why functions like `ord` (which converts a character to its numeric code) require a **Char**, not a String:
>
> - ✅ `ord 'A'` works (single character)
> - ❌ `ord "A"` doesn't work (that's a list containing one character)

Experiment

Try these expressions:

```
> 5 + 7
> 8 / 4
> True && False
> not (False && True)
> 7 == 7
> 5 /= 5
> 5 /= 6
> "hello" == "world"
> min 64 128
> max 128 64
> mod 14 3
> mod 14 3 == 0
> div 14 3
> max "clockwork" "orange"
> (2 == 3) || (2 == 2)
> ord 'A'
> chr 98
```

Questions

- What is the purpose of `&&` and `||`?
- What does `/=` mean?
- What is the significance of `ord` and `chr`?
- What do `mod` and `div` do? Try different values to help you decide.

Variables

In the REPL define some ***variables***:

```
> x = 7
> y = 2
> myValue = 17
> mySecondValue = -12
```

You can use these variables as values in expressions. Try the following, and some of your own:

```
> myValue + 3
> y - mySecondValue
> y == 7
> y == 2
> myValue < 15
> not (myValue > 15)
> myNewValue = x + y
> myNewValue + 11
```

What happens if you now try:

```
> myValue = 10
```

or

```
> y = 42
```

In functional programming variables and other data structures are ***immutable***. This means that the values stored in them cannot be changed after being created.

This is particularly important when working with ***Big Data***: Big Data is distributed across multiple devices, so immutability ensures that values stored cannot be changed after being created, preventing conflicts when multiple devices process the same data simultaneously.

Simple Functions

In addition to the built-in ***functions***, you can define your own.

In the REPL type the following:

```
> add x y = x + y
```

then try the following:

```
> add 4 5
> add (-6) 7
```

- `add` is the ***identifier*** (name of the function)
- `x` `y` are the ***parameters*** of the function
- `= x + y` defines what the function does

When we type `add (-6) 7`, the `(-6)` and `7` are the ***arguments*** that are passed into the **parameters** `x` and `y`.

> ***Note:*** *When passing negative numbers as arguments, always use parentheses, e.g.,* `(-6)` *not* `-6`*. This avoids confusion with the subtraction operator.*

Challenges

Now write functions to perform the following tasks:

- **`tripleMe`** - Accepts one parameter and returns three times its value

```
> tripleMe 5
15
```

- **`doubleAdd`** - Accepts two parameters, doubles each of them, then adds them together

```
> doubleAdd 3 4
14
```

- **`subtractThenDouble`** - Accepts two parameters, subtracts the second from the first, then doubles the result

```
> subtractThenDouble 10 3
14
```

- **`isGreaterThan100`** - Accepts two parameters, adds them together, and returns `True` if the sum is greater than 100

```
> isGreaterThan100 60 50
True
> isGreaterThan100 30 40
False
```

- **`averageTwo`** - Accepts two parameters and returns their average

```
> averageTwo 10 20
15
> averageTwo 7 8
7.5
```

- **`minutesToHours`** - Accepts a number of minutes and returns how many complete hours. Use `div` for integer division

```
> minutesToHours 150
2
> minutesToHours 90
1
```

> **`isEven`** - Accepts one parameter and returns `True` if the number is even, `False` otherwise. Use `mod` to find the remainder
>
> ```
> > isEven 8
> True
> > isEven 7
> False
> ```

Partial Function Application

In **mathematics**, a function maps a **single** value from its ***domain*** to a **single** value in its ***co-domain***. For example:

```
add(x, y) = x + y
```

This looks like a function taking two values, but in mathematics it is formally treated as taking one input, which is a pair of integers such as `(3,4)`. We write the ***type definition*** as:

```
add : ℤ × ℤ → ℤ
```

Functional programming languages use a different representation called ***currying***. In Haskish (and Haskell) we can define:

```
add x y = x + y
```

Haskish interprets this as: "`add` is a function that takes one parameter `x` and returns a new function that takes one parameter `y`."

Function application is the process of passing arguments to a function. When you write:

```
add 3 5
```

it is really:

```
(add 3) 5
```

because function application is left-associative. The steps are:

1. `add 3` creates a new function that adds 3 to its input
2. That new function is then applied to 5
3. The result is 8

Try this in the REPL:

```
> add x y = x + y
> add 3 5
8
> addThree = add 3
> addThree 5
8
> addThree 10
13
```

Notice how `addThree = add 3` creates a specialised function. This is called ***partial function application*** (or just ***partial application***) - applying some but not all of the arguments to create a new, more specialised function.

Challenges

Try creating some specialised functions through partial application.

> Create a function `subtract x y = x - y`, then use partial application to create `subtractFromHundred` (which subtracts its argument from 100).
>
> ```
> > subtractFromHundred 30
> 70
> > subtractFromHundred 5
> 95
> ```

> Create a function `addSuffix suffix text = text ++ suffix`, then use partial application to create `addExclamation` (which adds "!" to text)
>
> ```
> > addExclamation "Real horrorshow"
> "Real horrorshow!"
> ```

> Use the built-in `min` function with partial application to create `capAt100` (which ensures a value doesn't exceed 100)
>
> ```
> > capAt100 75
> 75
> > capAt100 150
> 100
> ```

> Create a function `addPrefix prefix text = prefix ++ text`, then use partial application to create `addMr` (which adds "Mr. " to names)
>
> ```
> > addMr "Spock"
> "Mr. Spock"
> > addMr "Benn"
> "Mr. Benn"
> ```

First-Class Functions

In the previous exercise, when you wrote `addTen = add 10`, you were assigning a function to a variable. This works because in functional programming languages, ***functions are first-class objects*** (also called **first-class values**).

What does "first-class object" mean?

A ***first-class object*** is a value that can be:

- **Assigned to a variable** - store it for later use
- **Passed as an argument** - give it to another function
- **Returned from a function** - be the result that a function gives back
- **Used in expressions** - appear anywhere a value can appear

Functions as Arguments

Functions can be passed to other functions. Here's a simple example - a function that applies another function twice:

```
> applyTwice f x = f (f x)
> double n = n * 2
> applyTwice double 3
12
```

Let's trace this: `applyTwice double 3` means `double (double 3)` = `double 6` = `12`

Functions as Return Values

Functions can return other functions (you've seen this with partial application!):

```
> makeAdder n = \x -> x + n
> addFive = makeAdder 5
> addTen = makeAdder 10
> addFive 3
8
> addTen 3
13
```

Challenges

Write a function `applyThrice` that takes a function and a value, and applies the function three times

```
> double n = n * 2
> applyThrice double 1
8
> addOne n = n + 1
> applyThrice addOne 0
3
```

Write a function `applyToFive` that takes a function as an argument and applies it to the number 5

```
> double n = n * 2
> applyToFive double
10
> square n = n * n
> applyToFive square
25
```

Write a function `applyBoth` that takes two functions `f` and `g`, and a value `x`, then returns `f x + g x`

```
> double n = n * 2
> triple n = n * 3
> applyBoth double triple 4
20
```

Function Composition

Now that you understand functions are first-class objects, you can learn a powerful way to combine them.

Function composition is the operation of combining two functions to create a new function. The output of one function becomes the input to another.

Mathematical Notation

Given two functions:

- `f: A → B` (takes values from set A, returns values in set B)
- `g: B → C` (takes values from set B, returns values in set C)

The ***composition*** `g ∘ f` (read as "g composed with f" or "g after f") creates a new function: `g ∘ f: A → C`

> **Important:** In `g ∘ f`, the function `f` is applied **first**, then `g` is applied to the result.

Function Composition in Haskish

In Haskish, we use the `.` operator for composition (instead of `∘`):

```
> addTwo x = x + 2
> cube x = x * x * x
> addThenCube = cube . addTwo
> addThenCube 3
125
```

Let's trace this:

- `addThenCube 3` = `(cube . addTwo) 3`
- First apply addTwo: `addTwo 3` = `5`
- Then apply cube: `cube 5` = `125`

> *Remember: functions are applied **right to left** (the rightmost function is applied first).*

Challenges

Define `addFive x = x + 5` and `double x = x * 2`, then create `addFiveThenDouble` using composition

```
> addFiveThenDouble 3
16
```

Define `square x = x * x` and `addThree x = x + 3`, then create `squareThenAddThree` using composition

```
> squareThenAddThree 4
19
```

The built-in `head` function gets the first element of a list, and `tail` gets all but the first. Create a function `secondElement` that gets the second element of a list using composition.

```
> secondElement [10,20,30,40]
20
> secondElement "hello"
'e'
```

Using the Function Panel

So far we've been writing functions in the REPL. For more complex programs, we use the ***Function Definition Panel***.

The function panel lets you:

- Write multiple functions that work together
- Save your work
- Build more complex programs using helper functions

Try writing these helper functions in the function panel:

```
square x = x * x
```

Click **Run Code** to make the function available in the REPL. Then test it:

```
> square 5
25
> square 12
144
```

Now add a function that uses `square` as a helper:

```
sumOfSquares a b = square a + square b
```

Challenge

Now write a complete program with helper functions:

`isRightTriangle` - Accepts three parameters (sides of a triangle) and returns `True` if they form a right-angled triangle using Pythagoras' theorem ($a^2 + b^2 = c^2$).

You'll need these helper functions:

- `square x` - returns x squared
- `sumOfSquares a b` - returns the sum of a^2 and b^2
- `isRightTriangle a b c` - checks if $a^2 + b^2 = c^2$

```
> isRightTriangle 3 4 5
True
> isRightTriangle 5 12 13
True
> isRightTriangle 1 2 3
False
```

Building More Complex Programs

Challenge

Now create a more complex program with multiple helper functions.

Write a program to analyse quadratic equations ($ax^2 + bx + c = 0$).

The **discriminant** tells us how many real solutions the equation has:

- discriminant = $b^2 - 4ac$
- If discriminant > 0, there are **two** real roots
- If discriminant = 0, there is **one** real root (repeated)
- If discriminant < 0, there are **no** real roots

Build these functions in the function panel:

- `discriminant a b c` - Calculates $b^2 - 4ac$
- `hasRealRoots a b c` - Returns `True` if the equation has at least one real root
- `hasTwoRoots a b c` - Returns `True` if the equation has two distinct real roots
- `hasOneRoot a b c` - Returns `True` if the equation has exactly one root

```
> discriminant 1 (-5) 6
1
> hasRealRoots 1 (-5) 6
True
> hasTwoRoots 1 (-5) 6
True
> hasOneRoot 2 4 2
True
> hasRealRoots 1 0 1
False
```

Introduction to Lists

Lists are collections of values of the same data type. They are enclosed in `[ ]`. Lists can be ***finite*** or ***infinite*** in length.

Try making some lists:

```
> [45,23,9,78]
> [8..15]
> [5..]
> [3,7..50]
> [5,3..-20]
> heroes = ["superman", "spiderman", "batman"]
```

Now try the following, taking note of any **errors**:

```
> head heroes
> tail heroes
> null heroes
> "thor" : heroes
> heroes : "thor"
> extraHeroes = "thing" : heroes
> heroesAndVillains = extraHeroes ++ ["joker","magneto"]
> "kingpin" ++ heroesAndVillains
> length heroesAndVillains
```

> ### *Note on Strings*
>
> *In Haskish (like Haskell),* ***String = [Char]****. This means strings are actually lists of characters!*
>
> *When you write* `"hello"`*, it's equivalent to* `['h','e','l','l','o']`
>
> *This is why you can use list functions like* `head`*,* `tail`*,* `map`*, and* `filter` *on strings!*

Questions

- What does `head` do?
- What does `tail` do?
- What is the purpose of `null`?
- Describe the purpose of the cons operator `:`
- What does `++` do?
- Why does `heroes : "thor"` throw an error?
- What is the purpose of `!!`?
- What does `take` do?
- What does `drop` do?

Pattern Matching & Recursion

Recursion allows a function to call itself with a simpler version of the problem. Every recursive function needs:

- A ***base case*** - when to stop recursing
- A ***recursive case*** - how to break down the problem

Example 1: Factorial

Factorial is defined mathematically as:

- 0! = 1 (base case)
- n! = n × (n-1)! (recursive case)

Define the factorial function in the function definition panel:

```
factorial 0 = 1
factorial n = n * factorial (n-1)
```

Trace the execution:

```
factorial 4
= 4 * factorial 3
= 4 * 3 * factorial 2
= 4 * 3 * 2 * factorial 1
= 4 * 3 * 2 * 1 * factorial 0
= 4 * 3 * 2 * 1 * 1
= 24
```

Example 2: Pattern Matching with Lists

Pattern matching lets you define functions by matching on the ***structure*** of data. Lists can be matched as either `[]` (empty) or `(x:xs)` (head and tail).

The `length` function can be defined recursively:

```
calcLength [] = 0
calcLength (x:xs) = 1 + calcLength xs
```

- **Base case:** An empty list `[]` has length 0
- **Recursive case:** A list `(x:xs)` has length 1 (for head `x`) plus the length of the tail `xs`

Challenges

Write similar recursive functions:

- `sumList` - adds all numbers in a list (base: empty list = 0)
- `productList` - multiplies all numbers in a list (base: empty list = 1)
- `replicateEach` - replicates each element in a list twice

```
> replicateEach [1,2,3]
[1,1,2,2,3,3]
> replicateEach "hi"
"hhii"
```

- `flatten` - flattens a list of lists into a single list

```
> flatten [[1,2],[3,4],[5,6]]
[1,2,3,4,5,6]
```

Guards & Conditionals

Haskish supports two main ways to make decisions in your code: ***Guards*** and ***Conditional expressions (if-then-else)***.

Conditional Expressions (if-then-else)

An `if-then-else` expression lets you choose between two values based on a condition. Both `then` and `else` branches are required.

```
countOccurrences e [] = 0
countOccurrences e (x:xs) = if e == x
                            then 1 + countOccurrences e xs
                            else countOccurrences e xs
```

Guards

Guards let you test multiple conditions in sequence. Each guard starts with `|` (pipe symbol) followed by a boolean condition:

```
countOccurrences e [] = 0
countOccurrences e (x:xs)
  | e == x    = 1 + countOccurrences e xs
  | otherwise = countOccurrences e xs
```

The `otherwise` keyword is just `True` - it catches any case that doesn't match the earlier guards.

When to use which:

- **if-then-else:** Simple binary choice between two options
- **Guards:** Multiple conditions or when the logic reads more naturally as a list of cases

Challenges

`switchBinary` - Given a list of binary digits (0s and 1s), return a new list with 0s changed to 1s and 1s changed to 0s

```
> switchBinary [0,1,1,0,1]
[1,0,0,1,0]
```

`inList` - Return `True` if a given element appears in the list, `False` otherwise

```
> inList 5 [1,3,5,7]
True
> inList 4 [1,3,5,7]
False
```

`removeAll` - Remove all occurrences of a given element from a list

```
> removeAll 2 [1,2,3,2,4,2]
[1,3,4]
```

`replace` - Replace all instances of one element with another element

```
> replace 2 9 [1,2,3,2,4]
[1,9,3,9,4]
```

Higher-Order Functions: Map, Filter, and Fold

A ***higher-order function*** is a function that either takes functions as arguments or returns functions as results (or both!).

Three higher-order functions form the backbone of functional programming:

- **`map`** - transforms every element in a list
- **`filter`** - keeps only elements that satisfy a condition
- **`fold`** - combines all elements into a single result

Map: Transforming Lists

`map` takes a function and a list, and applies that function to every element:

```
> double x = x * 2
> map double [1,2,3,4]
[2,4,6,8]

> square x = x * x
> map square [1,2,3,4,5]
[1,4,9,16,25]

> map (*3) [1,2,3,4]
[3,6,9,12]
```

Filter: Selecting Elements

`filter` takes a predicate (a function that returns `True` or `False`) and a list, keeping only elements where the predicate is `True`:

```
> isEven x = mod x 2 == 0
> filter isEven [1,2,3,4,5,6]
[2,4,6]

> filter (>5) [1,3,6,2,8,4,9]
[6,8,9]
```

Fold: Combining Elements

`fold` takes a combining function, an initial value, and a list, then combines all elements into a single result:

```
> fold (+) 0 [1,2,3,4,5]
15

> fold (*) 1 [1,2,3,4,5]
120

> fold max 0 [3,7,2,9,4]
9
```

Let's trace `fold (+) 0 [1,2,3]`:

1. Start with `0` (the initial value)
2. `0 + 1 = 1`
3. `1 + 2 = 3`
4. `3 + 3 = 6`
5. Result: `6`

Combining Map, Filter, and Fold

The real power comes from combining these functions:

```
> fold (+) 0 (map (*2) [1,2,3,4])
20

> fold (+) 0 (filter (>3) [1,2,3,4,5,6])
15

> map (*2) (filter isEven [1,2,3,4,5,6])
[4,8,12]
```

Challenges

Get the sum of squares of all even numbers in a list

```
> square x = x * x
> fold ... (map ... (filter ... [1,2,3,4,5,6]))
56
```

Use `fold` to count how many elements are in a list

```
> fold ... [10,20,30,40,50]
5
```

Extract the length of each string, then sum the lengths

```
> fold ... (map ... ["hello","world","functional"])
20
```

List Comprehensions

In mathematics, you've probably seen ***set-builder notation***:

```
{ x² | x ∈ ℕ, x < 10 }
```

List comprehensions in Haskish use almost identical syntax to describe lists.

Basic Syntax

The general form of a list comprehension is:

```
[ expression | variable <- list ]
```

Think of it as: "create a list by evaluating **expression** for each **variable** drawn from **list**"

Let's see how the mathematical set notation translates to Haskish:

Mathematics:

```
{ x² | x ∈ {1,2,3,4,5} }
```

Haskish:

```
> [x * x | x <- [1,2,3,4,5]]
[1,4,9,16,25]
```

Adding Predicates (Filters)

You can add conditions (predicates) to filter elements:

```
> [x * x | x <- [0..9], mod x 2 == 0]
[0,4,16,36,64]

> [x | x <- [1..20], mod x 2 == 0, x > 10]
[12,14,16,18,20]
```

Multiple Generators

List comprehensions can draw from multiple lists, creating combinations:

```
> [(x,y) | x <- [1,2,3], y <- [4,5]]
[(1,4),(1,5),(2,4),(2,5),(3,4),(3,5)]

> [x + y | x <- [1,2,3], y <- [10,20]]
[11,21,12,22,13,23]
```

Challenges

Create a list of the first 10 perfect squares (1, 4, 9, 16, ...)

```
> [... | x <- [1..10]]
[1,4,9,16,25,36,49,64,81,100]
```

Create a list of all numbers from 1 to 20 that are divisible by 3

```
> [x | x <- [1..20], ...]
[3,6,9,12,15,18]
```

Generate all Pythagorean triples (a,b,c) where $a^2 + b^2 = c^2$ and all values are between 1 and 20

```
> [(a,b,c) | a <- [1..20], b <- [1..20], c <- [1..20], a*a + b*b == c*c]
[(3,4,5),(4,3,5),(5,12,13),...]
```

Tuples and Destructuring

In functional programming, ***tuples*** are fixed-size collections that can hold values of different types.

Creating Tuples

Tuples are created using parentheses and commas:

```
> (1, 2)
(1,2)

> ("Alice", 25, True)
("Alice",25,True)

> ('x', "hello", 42)
('x',"hello",42)
```

Pattern Matching with Tuples (Destructuring)

The most powerful way to work with tuples is ***pattern matching*** (also called ***destructuring***):

```
> addPair (x, y) = x + y
> addPair (3, 5)
8

> fullName (first, last) = first ++ " " ++ last
> fullName ("Ada", "Lovelace")
"Ada Lovelace"
```

Tuples in Lists

You can have lists of tuples, which is extremely useful:

```
> pairs = [(1, 2), (3, 4), (5, 6)]

> firstElements xs = [x | (x, y) <- xs]
> firstElements [(1, 2), (3, 4), (5, 6)]
[1,3,5]

> sumPairs xs = [x + y | (x, y) <- xs]
> sumPairs [(1, 2), (3, 4), (5, 6)]
[3,7,11]
```

Challenges

`swap` - swaps the elements of a pair

```
> swap (1, 2)
(2,1)
```

`zip` - takes two lists and returns a list of pairs, combining corresponding elements

```
> zip [1, 2, 3] [4, 5, 6]
[(1,4),(2,5),(3,6)]
> zip "abc" [1, 2, 3]
[('a',1),('b',2),('c',3)]
```

`dotProduct` - computes the dot product of two vectors

```
> dotProduct [1, 2, 3] [4, 5, 6]
32
```

Lambda Functions

A ***lambda function*** is a function without a name. You create it on the spot when you need it, use it, and don't save it for later.

Lambda Syntax

In Haskish, lambda functions use the backslash `\` (which looks like the Greek letter λ "lambda"):

```
\parameter -> expression
```

The `\` starts the lambda, followed by the parameter(s), then `->`, then the expression that becomes the result.

Basic Examples

```
> (\x -> x * 2) 5
10

> map (\x -> x * 2) [1,2,3]
[2,4,6]

> map (\s -> s ++ "!") ["hello", "world"]
["hello!","world!"]
```

Lambdas with Map, Filter, and Fold

```
> map (\x -> x * x) [1,2,3,4]
[1,4,9,16]

> filter (\x -> mod x 2 == 0) [1,2,3,4,5,6]
[2,4,6]

> fold (\acc x -> acc + x) 0 [1,2,3,4]
10
```

Lambdas with Tuple Destructuring

Where lambdas really shine is when working with tuples:

```
> map (\(x,y) -> x + y) [(1,2), (3,4), (5,6)]
[3,7,11]

> filter (\(x,y) -> x > y) [(5,3), (2,8), (7,1)]
[(5,3),(7,1)]
```

When to use lambdas:

- Simple, one-time use functions
- Very short operations (like `\x -> x * 2`)
- Inline with `map`, `filter`, `fold`

When to use named functions:

- Complex logic that's hard to read in one line
- Functions you'll use multiple times
- Functions that deserve a descriptive name for clarity

Challenges

Use a lambda with `map` to convert a list of numbers to their negatives

```
> map (\x -> ...) [1,2,3,4,5]
[-1,-2,-3,-4,-5]
```

Use a lambda with `filter` to keep only strings longer than 3 characters

```
> filter (\s -> ...) ["hi", "hello", "bye", "world"]
["hello","world"]
```

Use `fold` with a lambda to count how many numbers in a list are greater than 5

```
> fold (\acc x -> ...) 0 [1,7,3,9,2,8]
3
```

Write `sumOfSquaresOfEvens` using only lambdas with `map`, `filter`, and `fold`

```
> sumOfSquaresOfEvens [1,2,3,4,5,6]
56
```

End of Haskish Programming Guide